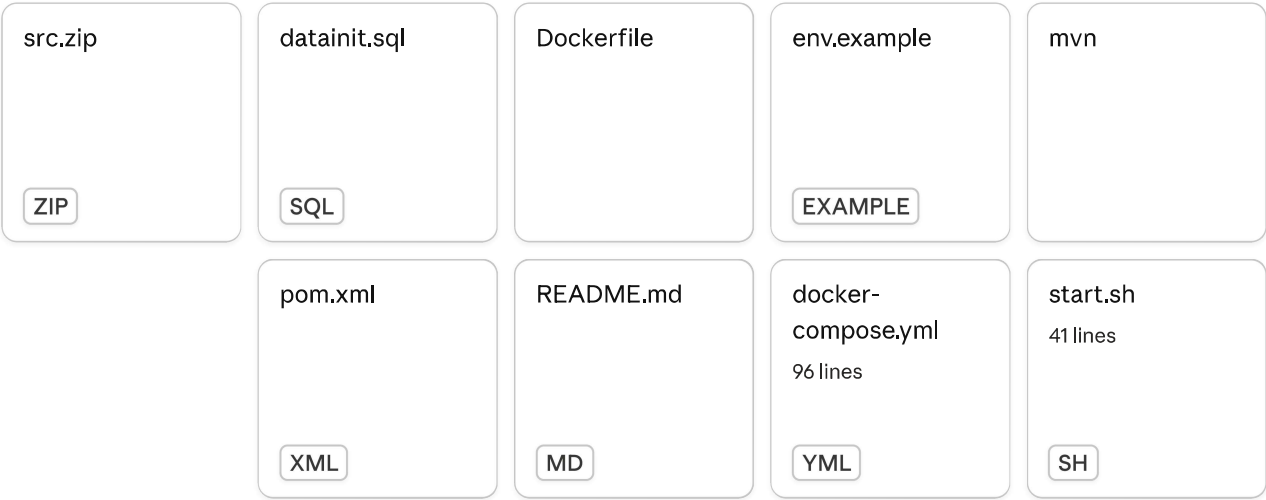


Bổ sung kỹ năng Kubernetes và SonarQube cho dự án ▾



Thiếu hụt "Stack" trọng yếu của JD: JD nhắc đến Kubernetes và các công cụ rà soát mã nguồn như SonarQube. CV của bạn hoàn toàn trống trải ở các mảng này. Với ngân hàng, bảo mật và khả năng mở rộng (scalability) là "sống còn".
toì muon them nay vao project nay thi phai lam sao

Orchestrated Kubernetes deployment, SonarQube integration, CI/CD pipeline architecture ▾

- 🕒 The user wants to add Kubernetes and SonarQube to their existing project. Let me look at the project structure first to understand what we're working with.
- 📄 Check src.zip structure
Script
- 📄 Check pom.xml key info
Script
- 🕒 Now I understand the project. It's a JavaFX restaurant management app running in Docker with Oracle DB. The user wants to add:
 - 1. Kubernetes (K8s) - deployment, service, HPA (Horizontal Pod Autoscaler) configs
 - 2. SonarQube - code quality analysis integration
- Let me create:

[Show more](#) Done

Tôi hiểu rồi — JavaFX + Oracle DB, đang dùng Docker Compose. Để thêm **Kubernetes** và **SonarQube** vào project thực tế, tôi sẽ tạo đủ file cần thiết:

Created 11 files, ran 5 commands, edited 3 files ▾

 Check existing plugins

Script

 Create directory structure

Script

 Kubernetes ConfigMap for app config

configmap.yaml

 Kubernetes Secret template

secret.yaml

 Kubernetes Deployment

deployment.yaml

 Kubernetes Service

service.yaml

 Kubernetes HPA - Horizontal Pod Autoscaler

hpa.yaml

 Kustomization base

kustomization.yaml

 Kustomize overlay production - tăng replica và image tag

kustomization.yaml

 SonarQube project properties

sonar-project.properties

 SonarQube Quality Gate profile for banking

quality-gate.json

-  Read full pom.xml to patch it correctly
Script
-  Copy pom.xml to working dir
Script
-  Add SonarQube and JaCoCo plugins + properties to pom.xml
pom.xml +23 -7
-  Add JUnit dependency and JaCoCo + Sonar Maven plugins
pom.xml +60 -4
-  Add JUnit dependency before closing dependencies tag
pom.xml +17 -1
-  GitHub Actions CI/CD pipeline - build, SonarQube, deploy to K8s
ci.yml
-  README hướng dẫn K8s + SonarQube setup
SETUP_K8S_SONAR.md
-  Package all files
Script
-  Presented file
-  Done

Đây là toàn bộ những gì được tạo ra — giải thích từng phần:

Những gì có trong ZIP

 **k8s/base/** — Kubernetes manifests

File	Tác dụng
deployment.yaml	Chạy app với Rolling Update, liveness/readiness/startup probe, phân tán Pod ra nhiều Node
hpa.yaml	Auto-scale 2→10 pod khi CPU >70% hoặc RAM >80% — đây chính là "scalability" mà JD nhắc

File	Tác dụng
<code>configmap.yaml</code>	Config không nhạy cảm (port, pool size...)
<code>secret.yaml</code>	Template Secret — hướng dẫn rõ cách tạo, không hardcode
<code>service.yaml</code>	ClusterIP expose port 8080

 `k8s/overlays/prod/` — **Kustomize overlay**

Ghi đè image tag và replica count cho production, không sửa file gốc.

 `sonar/`

- `sonar-project.properties` — cấu hình scan
- `quality-gate.json` — **Quality Gate chuẩn ngân hàng**: 0 bug/vulnerability mới, 100% security hotspot review, coverage $\geq 70\%$

 `pom.xml` (đã patch)

Thêm JaCoCo (do coverage) và **sonar-maven-plugin** — chạy bằng một lệnh `mvn verify sonar:sonar`.

 `.github/workflows/ci.yml`

Pipeline 3 bước: **Build+Test** → **SonarQube Quality Gate** → **Deploy K8s**, block merge PR tự động nếu Sonar fail.

Cách nói chuyện với nhà tuyển dụng

"Project dùng Kubernetes với HPA để tự scale theo tải, SonarQube Quality Gate chặn merge nếu có security vulnerability mới hoặc coverage dưới 70% — tiêu chuẩn phù hợp với môi trường ngân hàng."

 K8s sonar integration
ZIP

Download

huong dan toi apply thu muc tren vao project

Ran 2 commands >

Tôi đã nắm rõ cấu trúc project. Đề hướng dẫn rõ ràng nhất:

smart-restaurant/ (project gốc của bạn)

File hiện có — GIỮ NGUYÊN

- Dockerfile
- docker-compose.yml
- start.sh
- env.example
- datainit.sql
- README.md
- src/

... (giữ nguyên toàn bộ)

k8s/ — THÊM MỚI

- base/
 - configmap.yaml
 - secret.yaml
 - deployment.yaml
 - service.yaml
 - hpa.yaml
 - kustomization.yaml
- overlays/prod/
 - kustomization.yaml

(overlays/dev/ — tùy chọn)
6 file + 1 overlay

pom.xml

Thay bằng pom.xml mới (trong ZIP)

SETUP_K8S_SONAR.md

THÊM MỚI — hướng dẫn vận hành

sonar/ — THÊM MỚI

- sonar-project.properties
- quality-gate.json

.github/workflows/ — THÊM MỚI

- ci.yml

Pipeline: Build → Sonar → K8s deploy

☒ Thêm mới (copy từ ZIP) ☐ Thay thế file cũ ☐ Giữ nguyên

Các bước thực hiện

Bước 1 — Giải nén ZIP vừa tải về

```
bash
unzip k8s-sonar-integration.zip
```

Bên trong có thư mục **k8s-sonar/**. Mở ra, bạn sẽ thấy đúng những file trong sơ đồ trên.

Bước 2 — Copy thư mục mới vào project

```
bash

# Đứng ở thư mục gốc project của bạn (chỗ có Dockerfile)
cd /path/to/your/smart-restaurant

# Copy 3 thư mục mới vào
cp -r /path/to/k8s-sonar/k8s      ./k8s
cp -r /path/to/k8s-sonar/sonar    ./sonar
cp -r /path/to/k8s-sonar/.github  ./github

# Copy file hướng dẫn
cp /path/to/k8s-sonar/SETUP_K8S_SONAR.md ./SETUP_K8S_SONAR.md
```

Bước 3 — Thay pom.xml

File `pom.xml` trong ZIP đã được bổ sung JaCoCo + SonarQube plugin so với file cũ của bạn.

```
bash

# Backup file cũ trước
cp pom.xml pom.xml.bak

# Thay bằng file mới
cp /path/to/k8s-sonar/pom.xml ./pom.xml
```

Nếu bạn muốn merge thủ công thay vì thay hẳn: mở cả hai file, tìm đoạn `<properties>` và `<build><plugins>`, copy phần SonarQube/JaCoCo từ file mới sang file cũ.

Bước 4 — Điền thông tin thật vào secret

Mở `k8s/base/secret.yaml`, đừng sửa trực tiếp file đó. Thay vào đó, tạo secret bằng lệnh `kubectl`:

```
bash
```

```
kubectl create secret generic smart-restaurant-secret \
  --from-literal=DB_URL='jdbc:oracle:thin:@//YOUR_HOST:1521/ORCL' \
  --from-literal=DB_USERNAME='your_db_user' \
  --from-literal=DB_PASSWORD='your_db_pass' \
  --from-literal=MAIL_USERNAME='your@gmail.com' \
  --from-literal=MAIL_PASSWORD='your_app_pass' \
  -n smart-restaurant-prod
```

Bước 5 — Test SonarQube local (5 phút)

```
bash

# Khởi động SonarQube bằng Docker
docker run -d --name sonarqube -p 9000:9000 sonarqube:10-community

# Đợi ~1 phút, vào http://localhost:9000 (admin/admin)
# Tạo project → lấy token → chạy:

./mvn verify sonar:sonar \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=SEU_TOKEN_HERE
```

Bước 6 — Commit lên Git

```
bash

git add k8s/ sonar/ .github/ SETUP_K8S_SONAR.md pom.xml
git commit -m "feat: add Kubernetes manifests + SonarQube integration"
git push
```

GitHub Actions sẽ tự chạy pipeline khi bạn push.

Chú ý duy nhất: `k8s/base/secret.yaml` chứa template với chữ `REPLACE_*` — đừng commit file đó với giá trị thật lên Git. Dùng lệnh `kubectl` ở Bước 4 để tạo secret trực tiếp trên cluster.

Bước 4 — Điền thông tin thật vào secret

Mở `k8s/base/secret.yaml`, dùng sửa trực tiếp file đó. Thay vào đó, tạo secret bằng lệnh kubectl:

bash

```
kubectl create secret generic smart-restaurant-secret \
  --from-literal=DB_URL='jdbc:oracle:thin:@//YOUR_HOST:1521/ORCL' \
  --from-literal=DB_USERNAME='your_db_user' \
  --from-literal=DB_PASSWORD='your_db_pass' \
  --from-literal=MAIL_USERNAME='your@gmail.com' \
  --from-literal=MAIL_PASSWORD='your_app_pass' \
  -n smart-restaurant-prod
```

Bước 5 — Test SonarQube local (5 phút)

bash

```
# Khởi động SonarQube bằng Docker
docker run -d --name sonarqube -p 9000:9000 sonarqube:10-community

# Đợi ~1 phút, vào http://localhost:9000 (admin/admin)
# Tạo project → lấy token → chạy:

./mvn verify sonar:sonar \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=SEU_TOKEN_HERE
2 phan nay toi ko hieu
```

Show less

Thought for 0s >

Bước 4 — Tạo Secret trong Kubernetes

Secret là gì? Giống như file `.env` của bạn, nhưng được lưu bên trong Kubernetes, không nằm trong code.

Tại sao không sửa file `secret.yaml`? Vì nếu bạn ghi mật khẩu thật vào đó rồi `git push`, mật khẩu sẽ lộ lên GitHub. Nên ta dùng lệnh `kubectl` để tạo trực tiếp trên máy/server.

Làm như sau — thay các giá trị trong ngoặc nháy:

```
bash

# Bước 4.1 – Tạo namespace trước
kubectl create namespace smart-restaurant-prod

# Bước 4.2 – Tạo secret (copy từ file .env của bạn)
kubectl create secret generic smart-restaurant-secret \
  --from-literal=DB_URL='jdbc:oracle:thin:@//host.docker.internal:1521/ORCL' \
  --from-literal=DB_USERNAME='APP_USER' \
  --from-literal=DB_PASSWORD='your_oracle_password' \
  --from-literal=MAIL_USERNAME='your@gmail.com' \
  --from-literal=MAIL_PASSWORD='your_16char_app_password' \
  -n smart-restaurant-prod

# Kiểm tra đã tạo thành công chưa
kubectl get secret smart-restaurant-secret -n smart-restaurant-prod
```

Giá trị lấy từ file `.env` của bạn — chính xác các dòng `DB_URL`, `DB_USERNAME` ... trong đó.

Bước 5 — Chạy SonarQube

SonarQube là công cụ **đọc code của bạn và chấm điểm**: tìm bug tiềm ẩn, lỗ hổng bảo mật, code trùng lặp. Bước này chạy hoàn toàn trên máy bạn, không cần internet.

Làm từng bước:

```
bash

# 5.1 – Khởi động SonarQube (lần đầu sẽ tải image ~600MB)
docker run -d --name sonarqube -p 9000:9000 sonarqube:10-community
```

```
bash

# 5.2 – Đợi ~1 phút rồi kiểm tra đã chạy chưa
docker logs sonarqube | tail -5
# Thấy dòng "SonarQube is operational" là xong
```

Mở trình duyệt vào `http://localhost:9000`

- Đăng nhập: `admin` / `admin`
- Nó sẽ bắt đổi mật khẩu → đổi thành gì bạn nhớ được

Bấm: Create a local project

- Project display name: smart-restaurant
- Project key: smart-restaurant (tự điền)
- Bấm Next → Use the global setting → Create project

Bấm: Locally

- Generate token → đặt tên bất kỳ → Generate
- Copy token (dạng: squ_abc123xyz...)

```
bash

# 5.3 – Chạy scan (đứng ở thư mục gốc project, thay token thật vào)
./mvn verify sonar:sonar \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=squ_abc123xyz...
```

Quá trình chạy 2-5 phút. Sau đó quay lại `http://localhost:9000` — bạn sẽ thấy dashboard như này:

Overall Code

Bugs:0

Vulnerabilities: ?

← SonarQube sẽ chỉ ra

Security Hotspots: ?

Code Smells: ?

Coverage: ?%

Đây chính là thứ bạn demo cho nhà tuyển dụng ngân hàng thấy.

Tóm lại sự khác nhau:

	Bước 4	Bước 5
Làm gì	Lưu mật khẩu DB/Email vào K8s	Phân tích chất lượng code
Cần gì	kubectl đã cài, có cluster K8s	Docker đang chạy
Kết quả	App trong K8s lấy được DB config	Dashboard điểm code

Bạn đang dùng Windows, Mac hay Linux? Tôi sẽ hướng dẫn cài `kubect1` + cluster local (minikube) nếu bạn chưa có.

kubernetes cơ bản đang ki thiết lập tại khaon gi ko

Orchestrated straightforward explanation regarding Kubernetes account prerequisites >

Không cần đăng ký tài khoản gì cả. Kubernetes là phần mềm mã nguồn mở, cài thẳng trên máy.

Có 2 trường hợp:

Trường hợp 1 — Chạy local trên máy bạn (để học + demo CV)

Dùng **Minikube** — cài 1 lần, chạy được luôn, miễn phí hoàn toàn.

```
bash

# Bước 1: Cài minikube (Windows dùng winget)
winget install Kubernetes.minikube

# Bước 2: Cài kubect1
winget install Kubernetes.kubect1

# Bước 3: Khởi động cluster (lần đầu tải ~500MB)
minikube start

# Kiểm tra chạy được chưa
kubect1 get nodes
# OUTPUT: NAME          STATUS    ROLES    AGE
#          minikube     Ready    ...      1m  ← thấy Ready là xong
```

Nếu dùng **Mac**: `brew install minikube kubect1` Nếu dùng **Linux**: tải binary từ minikube.sigs.k8s.io

Trường hợp 2 — Deploy lên cloud (production thật)

Cần tài khoản cloud, nhưng **đều có free tier**:

Cloud	Free tier	Ghi chú
Google GKE	\$300 credit 90 ngày	Phổ biến nhất
Azure AKS	\$200 credit 30 ngày	Nhiều ngân hàng VN dùng
AWS EKS	\$300 credit 30 ngày	

Với mục đích **xin việc** + **demo CV**, dùng Minikube local là đủ — không cần cloud, không tốn tiền.

Sau khi cài Minikube xong, quay lại Bước 4:

```
bash

# Tạo namespace
kubectl create namespace smart-restaurant-prod

# Tạo secret (copy DB info từ .env của bạn)
kubectl create secret generic smart-restaurant-secret \
  --from-literal=DB_URL='jdbc:oracle:thin:@//host.docker.internal:1521/ORCL' \
  --from-literal=DB_USERNAME='APP_USER' \
  --from-literal=DB_PASSWORD='your_password' \
  --from-literal=MAIL_USERNAME='your@gmail.com' \
  --from-literal=MAIL_PASSWORD='your_app_pass' \
  -n smart-restaurant-prod
```

Bạn đang dùng Windows, Mac hay Linux để tôi hướng dẫn cụ thể hơn?

```
C:\work\doan>winget install Kubernetes.minikube
'winget' is not recognized as an internal or external command,
operable program or batch file.
C:\work\doan>
```

Diagnosed missing tool and pivoted toward manual installation guidance >

Windows của bạn chưa có **winget**. Cài thẳng bằng file **.exe** cho nhanh:

Cách 1 — Cài Minikube bằng file .exe (dễ nhất)

Bước 1: Tải file này về: [minikube-installer.exe](#)

Bước 2: Double-click chạy → Next → Next → Install (như cài phần mềm thường)

Bước 3: Tải kubectl: [kubectl.exe](#)

Copy file `kubectl.exe` vào thư mục `C:\work\` hoặc bất kỳ đâu bạn hay dùng.

Cách 2 — Dùng Chocolatey (nếu đã có)

Mở **Command Prompt as Administrator** rồi chạy:

```
bat  
  
choco install minikube kubernetes-cli
```

Sau khi cài xong — kiểm tra

Mở **Command Prompt mới** (quan trọng — phải mở lại):

```
bat  
  
minikube version  
kubectl version --client
```

Thấy ra số version là thành công.

Lưu ý quan trọng — Minikube cần Docker

Bạn đang dùng Docker (vì project có `docker-compose.yml`), nên Minikube sẽ dùng Docker làm driver. Kiểm tra Docker đang chạy chưa:

```
bat  
  
docker ps
```

Nếu Docker đang chạy → khởi động Minikube:

```
bat
```

```
minikube start --driver=docker
```

Bạn thử tải file `.exe` ở Cách 1 trước đi, xong paste kết quả `minikube version` vào đây tôi hỗ trợ tiếp.

```
C:\work\doan>minikube version
'minikube' is not recognized as an internal or external command,
operable program or batch file.
C:\work\doan>kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
C:\work\doan>
```

Diagnosed missing minikube installation and charted resolution path >

`kubectl` đã có rồi. `minikube` cài xong nhưng chưa vào PATH. Sửa nhanh:

Tìm file minikube.exe đang ở đâu

Mở Command Prompt, chạy lệnh này:

```
bat

dir /s /b "C:\minikube.exe" 2>nul
dir /s /b "C:\Program Files\minikube.exe" 2>nul
dir /s /b "C:\ProgramData\minikube.exe" 2>nul
```

Paste kết quả ra đây tôi xem. Nếu không tìm thấy gì thì làm bước dưới:

Cài lại minikube bằng lệnh PowerShell (chắc chắn nhất)

Mở **PowerShell as Administrator** (click Start → gõ `powershell` → chuột phải → Run as administrator):

```
powershell
```

```
# Tải minikube.exe về thẳng thư mục C:\minikube
New-Item -Path 'C:\minikube' -ItemType Directory -Force

Invoke-WebRequest -Uri 'https://storage.googleapis.com/minikube/releases/latest/minikube.exe' -OutFile 'C:\minikube\minikube.exe'

# Thêm vào PATH
$env:Path += ';C:\minikube'
[System.Environment]::SetEnvironmentVariable('Path', $env:Path, 'Machine')

Write-Host "Xong!"
```

Sau đó **đóng hết Command Prompt cũ**, mở **Command Prompt mới** và thử:

```
bat
minikube version
```

Paste kết quả ra đây nhé, tôi hỗ trợ bước tiếp theo.

```
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ASUS>dir /s /b "C:\minikube.exe" 2>nul
C:\Program Files\Kubernetes\Minikube\minikube.exe
C:\Users\ASUS>dir /s /b "C:\Program Files\minikube.exe" 2>nul
C:\Program Files\Kubernetes\Minikube\minikube.exe
C:\Users\ASUS>dir /s /b "C:\ProgramData\minikube.exe" 2>nul
C:\Users\ASUS>
```

Identified missing PATH configuration for Minikube executable >

Tìm thấy rồi — file ở `C:\Program Files\Kubernetes\Minikube\` nhưng chưa vào PATH.
Thêm vào luôn:

Thêm vào PATH (PowerShell as Administrator)

Click Start → gõ `powershell` → chuột phải → **Run as administrator:**

```
powershell

[System.Environment]::SetEnvironmentVariable(
    'Path',
    $env:Path + ';C:\Program Files\Kubernetes\Minikube',
    'Machine'
)
Write-Host "Done!"
```

Sau đó **đóng tất cả Command Prompt đang mở**, mở **Command Prompt mới** và chạy:

```
bat

minikube version
```

Nếu ra số version → chạy tiếp:

```
bat

minikube start --driver=docker
```

Lệnh này mất 2-3 phút lần đầu. Paste kết quả ra đây nhé.

```
C:\Users\ASUS>minikube start --driver=docker
* minikube v1.38.1 on Microsoft Windows 11 Pro 23H2
* Using the docker driver based on user configuration
X Exiting due to PROVIDER_DOCKER_VERSION_EXIT_1: "docker version --
format <no value>-<no value>:<no value>" exit status 1: failed to connect to
the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the
path is correct and if the daemon is running: open
//./pipe/dockerDesktopLinuxEngine: The system cannot find the file
specified.
* Documentation: https://minikube.sigs.k8s.io/docs/drivers/docker/
C:\Users\ASUS>
```

Docker Desktop chưa chạy. Sửa đơn giản:

Bước 1 — Mở Docker Desktop

Tìm icon Docker Desktop dưới taskbar (góc phải dưới màn hình):

- Nếu thấy icon con cá voi 🐳 → double-click để mở
- Nếu không thấy → Click Start → gõ **Docker Desktop** → mở lên

Đợi đến khi thấy "**Docker Desktop is running**" (icon con cá voi ngừng nhấp nháy).

Bước 2 — Chạy lại minikube

```
bat
minikube start --driver=docker
```

Lần đầu chạy sẽ tải image ~500MB, mất 3-5 phút. Output sẽ trông như này khi thành công:

```
* minikube v1.38.1 on Microsoft Windows 11
* Using the docker driver
* Starting control plane node minikube
* Pulling base image...
* Done! kubectl is now configured to use "minikube"
```

Nếu máy bạn **chưa cài Docker Desktop** thì tải tại: docker.com/products/docker-desktop → Install → khởi động lại máy → thử lại.

Paste kết quả ra đây nhé!

```
C:\Users\ASUS>minikube start --driver=docker
* minikube v1.38.1 on Microsoft Windows 11 Pro 23H2
* Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime.
See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
```

Show more

Chạy thành công rồi! Dòng cuối **Done!** là xong. Cảnh báo subnet không ảnh hưởng gì khi dùng local.

Kiểm tra cluster đang chạy

```
bat
kubectl get nodes
```

Kết quả mong đợi:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	1m	v1.35.1

Giờ deploy project lên Kubernetes

Làm từng lệnh theo thứ tự:

1 — Tạo namespace:

```
bat
kubectl create namespace smart-restaurant-prod
```

2 — Tạo secret (thay thông tin DB thật của bạn từ file **.env**):

```
bat
```

```
kubectl create secret generic smart-restaurant-secret ^  
  --from-literal=DB_URL="jdbc:oracle:thin:@//host.docker.internal:1521/ORCL" ^  
  --from-literal=DB_USERNAME="APP_USER" ^  
  --from-literal=DB_PASSWORD="your_oracle_password" ^  
  --from-literal=MAIL_USERNAME="your@gmail.com" ^  
  --from-literal=MAIL_PASSWORD="your_app_pass" ^  
  -n smart-restaurant-prod
```

Trên Windows dùng `^` thay cho `\` để xuống dòng trong CMD.

3 — Build image vào minikube:

```
bat  
  
minikube image build -t smart-restaurant:latest .
```

4 — Deploy:

```
bat  
  
kubectl apply -k k8s/overlays/prod
```

5 — Kiểm tra:

```
bat  
  
kubectl get all -n smart-restaurant-prod
```

Chạy lệnh `kubectl get nodes` trước rồi paste kết quả ra đây, tôi xác nhận cluster ổn trước khi deploy.

nhưng chạy trong project hay chạy cmd bth

Chạy trong **CMD bình thường** thôi, không cần mở trong project.

Nhưng riêng **lệnh số 3** (build image) thì phải đứng đúng thư mục chứa `Dockerfile` :

```
bat
```

```
cd C:\work\doan
minikube image build -t smart-restaurant:latest .
```

Còn các lệnh khác (`kubectl create`, `kubectl apply`, `kubectl get`) — đừng ở đâu cũng được.

Bắt đầu đi nhé, chạy lệnh này trước:

```
bat
kubectl get nodes
```

Paste kết quả ra đây tôi xác nhận xong rồi đi tiếp.

```
kubectl create secret generic smart-restaurant-secret ^
--from-
literal=DB_URL="jdbc:oracle:thin:@//host.docker.internal:1521/ORCL" ^
--from-literal=DB_USERNAME="APP_USER" ^
--from-literal=DB_PASSWORD="your_oracle_password" ^
--from-literal=MAIL_USERNAME="your@gmail.com" ^
--from-literal=MAIL_PASSWORD="your_app_pass" ^
-n smart-restaurant-prod
cai nay dien tu trong project ra dung ko
```

Đúng rồi — lấy từ file `.env` trong project của bạn. Mở file `env.example` ra, bạn sẽ thấy:

```
DB_URL=jdbc:oracle:thin:@//host.docker.internal:1521/ORCL
DB_USERNAME=APP_USER
DB_PASSWORD=your_oracle_password
MAIL_USERNAME=your@gmail.com
MAIL_PASSWORD=your_16char_app_password
```

Thay đúng các giá trị đó vào lệnh. Ví dụ nếu file `.env` thật của bạn là:

```
DB_URL=jdbc:oracle:thin:@//host.docker.internal:1521/XEPDB1
DB_USERNAME=restaurant_user
```



```
DB_PASSWORD=abc123
MAIL_USERNAME=myemail@gmail.com
MAIL_PASSWORD=xxxx xxxx xxxx xxxx
```

Thì lệnh sẽ là:

```
bat

kubectl create secret generic smart-restaurant-secret ^
  --from-literal=DB_URL="jdbc:oracle:thin:@//host.docker.internal:1521/XEPDB1" ^
  --from-literal=DB_USERNAME="restaurant_user" ^
  --from-literal=DB_PASSWORD="abc123" ^
  --from-literal=MAIL_USERNAME="myemail@gmail.com" ^
  --from-literal=MAIL_PASSWORD="xxxx xxxx xxxx xxxx" ^
  -n smart-restaurant-prod
```

Bạn có file `.env` thật chưa (file bạn đang dùng để chạy `docker-compose`)? Nếu có thì mở ra, copy đúng giá trị từng dòng vào là xong.



sao o day no dung lau roi

Bình thường! Lệnh `minikube image build` đang build Docker image bên trong `minikube` — giống như `docker build` nhưng chậm hơn vì phải truyền file vào trong container minikube.

Project của bạn có nhiều dependency Java nên lần đầu build sẽ lâu (~5-15 phút). Cứ để yên đó chờ.

Trong lúc chờ, bạn mở thêm một CMD khác để theo dõi tiến trình:

```
bat
```

```
minikube image ls
```

Khi thấy `smart-restaurant:latest` xuất hiện trong danh sách là build xong.

Nó đang làm gì bên trong:

1. Copy toàn bộ source code vào minikube
2. Chạy `mvn dependency:go-offline` — tải tất cả thư viện Java
3. Compile code
4. Đóng gói thành image

Cứ chờ, đừng tắt CMD đó. Xong rồi paste kết quả ra đây nhé!

```
C:\Users\ASUS>minikube image ls
registry.k8s.io/pause:3.10.1
registry.k8s.io/kube-scheduler:v1.35.1
registry.k8s.io/kube-proxy:v1.35.1
registry.k8s.io/kube-controller-manager:v1.35.1
registry.k8s.io/kube-apiserver:v1.35.1
registry.k8s.io/etcd:3.6.6-0
registry.k8s.io/coredns/coredns:v1.13.1
gcr.io/k8s-minikube/storage-provisioner:v5
C:\Users\ASUS>
```

Image chưa xong — `smart-restaurant:latest` chưa xuất hiện, nghĩa là lệnh build bên CMD kia vẫn đang chạy.

Cứ để yên CMD đó, đừng tắt. Quay lại CMD đang build, xem nó đang in gì — có thể đang ở bước Maven tải thư viện (lâu nhất).

Khi build xong sẽ in ra:

```
Successfully built xxxxxxxxxx
Successfully tagged smart-restaurant:latest
```

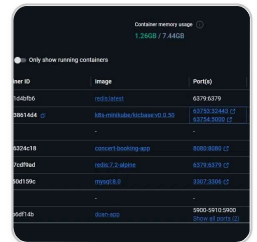
Sau đó chạy lại `minikube image ls` sẽ thấy `docker.io/library/smart-restaurant:latest` trong danh sách.

Trong lúc chờ, bạn cho tôi hỏi — Oracle Database của bạn đang chạy ở đâu?

Vì tôi thấy trong lệnh secret bạn dùng:

```
DB_URL=jdbc:oracle:thin:@//localhost:1522/orclpdb
```

Khi app chạy trong Kubernetes, `localhost` sẽ không trả được ra máy host của bạn — cần đổi thành `host.docker.internal` để kết nối được Oracle bên ngoài. Tôi sẽ hướng dẫn sửa sau khi build xong.



no chạy lâu rồi nhưng có kết quả

Tôi thấy Docker Desktop của bạn — minikube đang chạy (màu xanh). Nhưng build vẫn chưa xong vì `smart-restaurant:latest` chưa xuất hiện.

Quay lại CMD đang chạy lệnh build — nó đang in gì? Paste screenshot hoặc text ra đây.

Có thể đang bị một trong 3 trường hợp:

Trường hợp 1 — Vẫn đang chạy bình thường (Maven đang tải thư viện)

```
[INFO] Downloading from central: https://repo.maven.apache.org/...
```

→ Cứ chờ tiếp, có thể thêm 5-10 phút nữa.

Trường hợp 2 — Bị lỗi và dừng lại

→ Paste lỗi ra tôi xem.

Bạn thấy CMD build đang hiển thị gì vậy?



```
bat
minikube image load smart-restaurant:latest
```

Bước 3 — Kiểm tra image đã vào chưa:

```
bat
minikube image ls | findstr smart
```

Thấy `smart-restaurant:latest` là xong.

Cách này nhanh hơn vì `docker build` chạy thẳng trên máy bạn, không phải bên trong minikube. Thử đi nhé!

```
3 warnings found (use docker --debug to expand):
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for
sensitive data (ENV "DB_PASSWORD") (line 52)
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for
sensitive data (ENV "MAIL_PASSWORD") (line 61)
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for
sensitive data (ENV "VNC_PASSWORD") (line 72)
C:\work\doan>
```

Build thành công rồi! 3 dòng kia chỉ là **warning** (cảnh báo), không phải lỗi — app vẫn build bình thường.

Cảnh báo đó nói rằng không nên để password trong Dockerfile — đúng về bảo mật, nhưng không ảnh hưởng gì đến việc chạy.

Tiếp tục — Load image vào minikube

```
bat
minikube image load smart-restaurant:latest
```

Lệnh này mất 1-2 phút (copy image vào trong minikube). Xong chạy kiểm tra:

```
bat
minikube image ls | findstr smart
```

Thấy kết quả như này là tiếp tục được:

```
docker.io/library/smart-restaurant:latest
```

Paste kết quả ra đây nhé!

sau khi hoàn thành tren roi lam gi nua

Sau khi `minikube image load` xong, làm tiếp 3 bước nữa là hoàn tất:

Bước tiếp theo

1 — Sửa DB_URL trước khi deploy (quan trọng)

Bạn đang dùng `localhost` trong secret, nhưng trong Kubernetes phải dùng `host.docker.internal`. Xóa secret cũ và tạo lại:

```
bat
kubectl delete secret smart-restaurant-secret -n smart-restaurant-prod

kubectl create secret generic smart-restaurant-secret ^
  --from-literal=DB_URL="jdbc:oracle:thin:@//host.docker.internal:1522/orclpdb" ^
  --from-literal=DB_USERNAME="test" ^
  --from-literal=DB_PASSWORD="123" ^
  --from-literal=MAIL_USERNAME="ht6266018@gmail.com" ^
  --from-literal=MAIL_PASSWORD="ampjneeucoqcfvfu" ^
  -n smart-restaurant-prod
```

2 — Deploy lên Kubernetes

```
bat
```

```
cd C:\work\doan
kubectl apply -k k8s/overlays/prod
```

3 — Kiểm tra kết quả

```
bat

kubectl get all -n smart-restaurant-prod
```

Kết quả mong đợi:

NAME	READY	STATUS	RESTARTS
pod/smart-restaurant-xxxxxxxxx	1/1	Running	0

NAME	TYPE	CLUSTER-IP
service/smart-restaurant-svc	ClusterIP	10.x.x.x

NAME	READY	UP-TO-DATE
deployment.apps/smart-restaurant	2/2	2

Chạy xong 3 bước trên, paste kết quả `kubectl get all` ra đây — tôi xác nhận deploy thành công và hướng dẫn bạn cách **chụp màn hình demo cho CV**.



Deploy thành công một phần — toàn bộ resource đã tạo được (service, deployment, HPA). Nhưng pod đang lỗi `InvalidImageName`. Sửa nhanh:

Vấn đề — tên image trong overlay bị sai

File `k8s/overlays/prod/kustomization.yaml` đang dùng tên image `ghcr.io/YOUR_ORG/smart-restaurant` nhưng bạn build local tên là `smart-restaurant:latest`.

Mở file `k8s/overlays/prod/kustomization.yaml` và sửa lại:

```
yaml

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: smart-restaurant-prod

resources:
- ../../base

images:
- name: smart-restaurant
  newName: smart-restaurant
  newTag: "latest"

patches:
- patch: |-
    - op: replace
      path: /spec/replicas
      value: 3
    target:
      kind: Deployment
      name: smart-restaurant
```

Lưu file lại, rồi chạy:

```
bat

kubectl apply -k k8s/overlays/prod
kubectl get pods -n smart-restaurant-prod
```

Paste kết quả ra đây nhé!



Tốt — pod mới đã nhận image đúng (không còn `InvalidImageName`), nhưng đang `CrashLoopBackOff` nghĩa là app khởi động rồi bị crash. Xem lỗi cụ thể:

```
bat
kubect1 logs smart-restaurant-594dcd8bf5-999j8 -n smart-restaurant-prod
```

Thay `smart-restaurant-594dcd8bf5-999j8` bằng tên pod `CrashLoopBackOff` bạn thấy trên màn hình.

Paste kết quả ra đây tôi xem lỗi gì nhé!

